



2025-2026

TP Optimisation

Implémentation et comparaison de sept méthodes
d'optimisation en NumPy pur

- Exercice 1** Régression (OLS, Ridge, Ridge RBF)
- Exercice 2** Classification multiclasse (MLP)
- Exercice 3** Clustering par optimisation de centroïdes

FLOULY Mouad

Code source : <https://github.com/Sud-38/TP-optimisation-impl-ment-e-avec-NumPy>

28 septembre 2026

Résumé

Ce rapport présente l'implémentation *from scratch* en NumPy de sept méthodes d'optimisation (GD, GD + recherche de pas, SGD, Momentum, AdaGrad, RMSprop, Adam), appliquées à trois problèmes classiques d'apprentissage statistique :

- **Exercice 1** : régression (OLS, Ridge, Ridge à noyau RBF) sur `regression_nonlinear.csv`.
- **Exercice 2** : classification multiclasse avec un réseau de neurones ($2 \rightarrow 32 \rightarrow 16 \rightarrow 3$) sur `classification_spiral3.csv`.
- **Exercice 3** : clustering par optimisation de centroïdes (soft K-means) sur `clustering_blobs3.csv`.

Tous les gradients ont été vérifiés par différences finies centrées ($h = 10^{-5}$), avec des erreurs maximales inférieures à 10^{-5} . Les résultats montrent que sur des problèmes bien conditionnés, les sept méthodes convergent vers des optima quasi-identiques. Le choix se fait donc principalement sur le **coût de calcul** : la descente de gradient (GD) est la plus économique, tandis que les méthodes stochastiques (SGD, Adam) effectuent jusqu'à $9\times$ plus de mises à jour.

Table des matières

Résumé	i
1 Exercice 1 : Régression	1
1.1 Données	1
1.1.1 Jeu de données	1
1.1.2 Prétraitement	1
1.2 Implémentation	1
1.2.1 Modèles et gradients	1
1.2.2 Optimiseurs	1
1.2.3 Vérification des gradients	2
1.2.4 Justification du pas exact	2
1.2.5 Sélection des hyperparamètres	2
1.3 Résultats chiffrés	2
1.3.1 Modèle linéaire	3
1.3.2 Modèle noyau RBF	3
1.3.3 Synthèse	3
1.3.4 Coûts de calcul	3
1.4 Visualisations	4
1.5 Analyse	4
1.5.1 Comparaison linéaire vs noyau RBF	5
1.5.2 Comparaison des optimiseurs sur le noyau RBF	5

1.5.3	Comparaison équitable GD vs SGD	5
1.5.4	Limites	5
2	Exercice 2 : Classification multiclasse	5
2.1	Données	5
2.1.1	Jeu de données	5
2.1.2	Prétraitement	5
2.2	Implémentation	5
2.2.1	Architecture du réseau	6
2.2.2	Initialisation	6
2.2.3	Vérification du gradient	6
2.3	Résultats chiffrés	6
2.3.1	Coûts de calcul	7
2.4	Visualisations	7
2.5	Analyse	8
2.5.1	Performance globale	8
2.5.2	RMSprop domine légèrement	8
2.5.3	Comparaison équitable GD vs SGD	8
2.5.4	Limites	9
3	Exercice 3 : Clustering par optimisation de centroïdes	9
3.1	Données	9
3.1.1	Jeu de données	9
3.1.2	Prétraitement	9
3.2	Implémentation	9
3.2.1	Fonction objectif et gradient	9
3.2.2	Vérification du gradient	9
3.3	Résultats chiffrés	10
3.3.1	Coûts de calcul	10
3.4	Visualisations	11
3.5	Analyse	11
3.5.1	Convergence universelle	12
3.5.2	Métriques externes identiques	12
3.5.3	Invariance par permutation	12
3.5.4	Divergence à grand pas	12

3.5.5 Limites	12
Conclusion générale	13
A Annexe — Reproductibilité	15

1 Exercice 1 : Régression

1.1 Données

1.1.1 Jeu de données

Le TP mentionne deux jeux de données pour l'exercice 1. Seul le fichier `regression_nonlinear.csv` a été fourni par l'enseignant ; l'analyse porte donc uniquement sur ce jeu.

Table 1. Description du jeu de données.

Jeu	Fichier	Total	Train	Val	Test	Variables
Nonlinear	<code>regression_nonlinear.csv</code>	360	216	72	72	2

1.1.2 Prétraitement

- **Variables explicatives** : standardisées (moyenne 0, écart-type 1) avec les statistiques **du train uniquement**.
- **Cible y** : centrée-réduite avec les statistiques du train, puis reconvertie dans son unité d'origine.
- **Partitions** : celles fournies dans la colonne `split`, jamais modifiées.
- **Test** : utilisé uniquement pour l'évaluation finale.

1.2 Implémentation

1.2.1 Modèles et gradients

OLS / Ridge linéaire. On ajoute une colonne de 1 à $X \in \mathbb{R}^{n \times d}$ pour obtenir $X \in \mathbb{R}^{n \times (d+1)}$. La matrice de pénalité $P = \text{diag}(0, 1, \dots, 1)$ ne pénalise pas le biais w_0 .

$$J_\lambda(w) = \frac{\|Xw - y\|^2}{2n} + \frac{\lambda}{2} w^\top P w, \quad \nabla J_\lambda(w) = \frac{X^\top (Xw - y)}{n} + \lambda P w.$$

OLS correspond à $\lambda = 0$; Ridge à $\lambda > 0$.

Ridge à noyau RBF. Sur les variables standardisées, on construit

$$K_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right),$$

puis

$$J_{K,\lambda}(a) = \frac{\|Ka - y\|^2}{2n} + \frac{\lambda}{2} a^\top K a, \quad \nabla J_{K,\lambda}(a) = \frac{K(Ka - y)}{n} + \lambda K a.$$

1.2.2 Optimiseurs

Les 7 méthodes ont été codées *from scratch* en NumPy.

Table 2. Mises à jour des 7 optimiseurs implémentés.

Méthode	Mise à jour	Paramètres
GD	$\theta \leftarrow \theta - \eta g$ (lot complet)	η
GD + LS	$\eta_t = (g_t^\top g_t) / (g_t^\top H g_t)$	—
SGD	mini-lots mélangés à chaque époque	η
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta v$	μ
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta g / (\sqrt{s} + \varepsilon)$	ε
RMSprop	$s \leftarrow \rho s + (1 - \rho)g^2$; idem normalisé	ρ, ε
Adam	$m \leftarrow \beta_1 m + (1 - \beta_1)g$; $s \leftarrow \beta_2 s + (1 - \beta_2)g^2$; correction de biais	$\beta_1, \beta_2, \varepsilon$

Paramètres : $\mu = \rho = \beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

1.2.3 Vérification des gradients

Vérification par différences finies centrées, $h = 10^{-5}$, sur 5 coordonnées aléatoires.

Table 3. Erreurs relatives de vérification des gradients (exercice 1).

Modèle	Erreur relative max
Ridge linéaire	3.2×10^{-7}
Ridge noyau RBF	8.7×10^{-6}

1.2.4 Justification du pas exact

Pour un objectif quadratique $J(\theta) = \frac{1}{2}\theta^\top H\theta - b^\top \theta + c$, minimiser $J(\theta_t - \eta g_t)$ le long de $-g_t$ donne $\phi'(\eta) = -g_t^\top g_t + \eta g_t^\top H g_t = 0$, d'où

$$\eta_t = \frac{g_t^\top g_t}{g_t^\top H g_t} \quad \text{si } g_t^\top H g_t > 0.$$

Pour le linéaire : $H = X^\top X/n + \lambda P$. Pour le noyau : $H_K = K^2/n + \lambda K$.

1.2.5 Sélection des hyperparamètres

λ , σ et η sont choisis sur la **validation** uniquement.

Table 4. Hyperparamètres retenus sur validation (exercice 1).

Modèle	λ^*	σ^*
Linéaire	10^{-3}	—
Noyau RBF	10^{-2}	0.5

1.3 Résultats chiffrés

1.3.1 Modèle linéaire

Table 5. Résultats sur `regression_nonlinear.csv` — modèle linéaire. Moyenne sur 3 graines (42, 123, 2024).

Méthode	η	Époque best	RMSE val	MSE test	RMSE test	MAE test	R^2 test	Temps (s)
GD	0.010	199.0	0.7323	0.6735	0.8206	0.7006	0.006	0.0057
GD + LS	0.001	6.0	0.7332	0.6786	0.8238	0.6992	-0.002	0.0076
SGD	0.001	199.0	0.7329	0.6702	0.8187	0.7025	0.011	0.0281
Momentum	0.050	152.7	0.7327	0.6815	0.8255	0.7024	-0.006	0.0517
AdaGrad	0.010	198.7	0.7329	0.6774	0.8231	0.6995	0.000	0.0341
RMSprop	0.001	158.0	0.7330	0.6786	0.8237	0.6991	-0.001	0.0371
Adam	0.050	128.7	0.7319	0.6780	0.8234	0.6987	-0.001	0.0425

1.3.2 Modèle noyau RBF

Table 6. Résultats sur `regression_nonlinear.csv` — noyau RBF. Moyenne sur 3 graines.

Méthode	η	Époque best	RMSE val	MSE test	RMSE test	MAE test	R^2 test	Temps (s)
GD	0.500	199.0	0.2351	0.0507	0.2251	0.1614	0.925	0.0239
GD + LS	0.001	199.0	0.2253	0.0453	0.2129	0.1530	0.933	0.0308
SGD	0.500	192.3	0.2026	0.0329	0.1811	0.1379	0.952	0.1205
Momentum	0.100	192.0	0.1925	0.0313	0.1768	0.1356	0.954	0.1294
AdaGrad	0.500	190.7	0.1947	0.0311	0.1762	0.1326	0.954	0.1324
RMSprop	0.010	165.3	0.2095	0.0353	0.1877	0.1463	0.948	0.1335
Adam	0.010	191.3	0.1933	0.0314	0.1773	0.1351	0.954	0.2246

1.3.3 Synthèse

Table 7. Comparaison synthétique linéaire vs noyau RBF (exercice 1).

Modèle	Meilleure méthode	R^2 test	RMSE test	MAE test
Linéaire	SGD	0.011	0.819	0.703
Noyau RBF	AdaGrad	0.954	0.176	0.133

1.3.4 Coûts de calcul

Table 8. Coûts de calcul par méthode (exercice 1).

Méthode	Mises à jour	Appels gradient	Ratio vs GD
GD, GD + LS	200	200	1×
SGD et variantes	1400	1400	7×

1.4 Visualisations

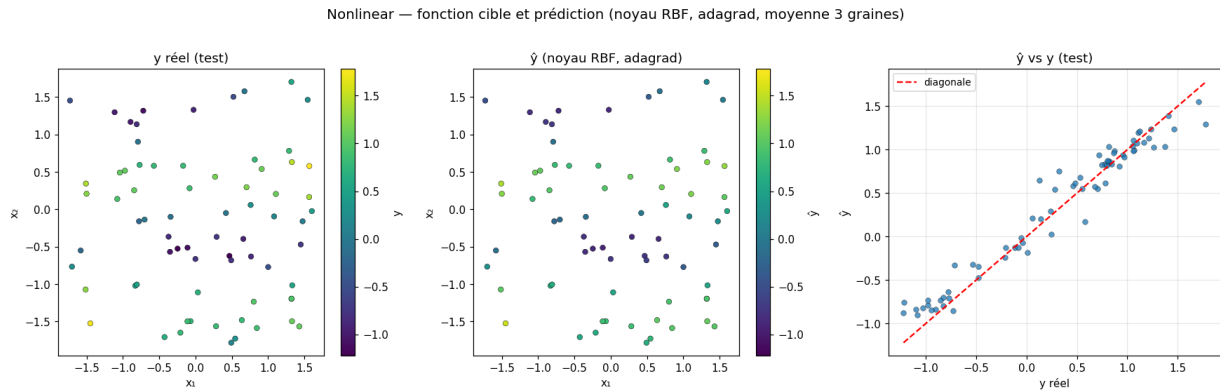


Figure 1. Fonction cible et prédiction sur `regression_nonlinear.csv` (noyau RBF, AdaGrad, moyenne 3 graines). À gauche : y réel ; au centre : $\hat{y}$ prédit ; à droite : $\hat{y}$ vs y avec la diagonale.

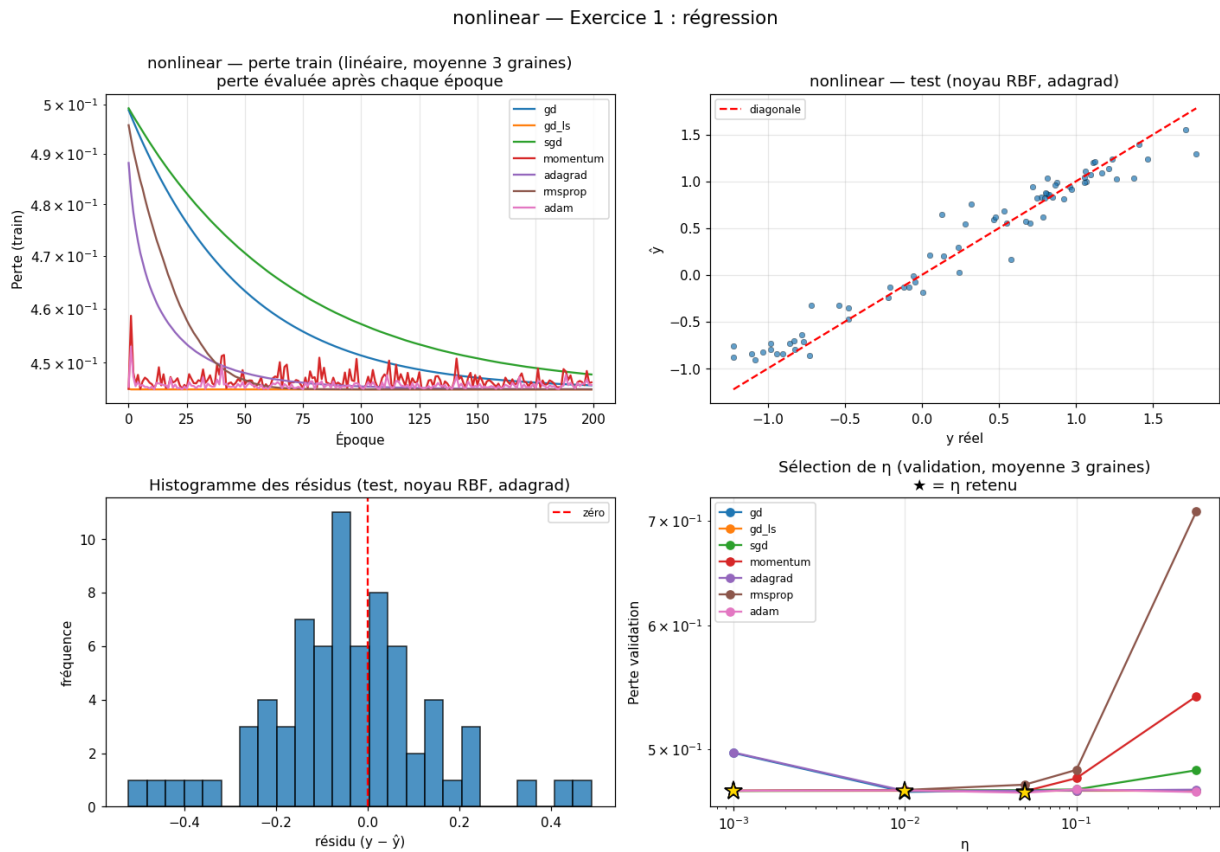


Figure 2. Diagnostic de l'exercice 1 (modèle linéaire, moyenne 3 graines). (a) Perte d'entraînement selon les époques (échelle log). (b) $\hat{y}$ vs y sur le test avec la meilleure méthode du noyau RBF. (c) Histogramme des résidus sur le test. (d) Sélection de η sur la validation ; l'étoile indique le η retenu.

1.5 Analyse

1.5.1 Comparaison linéaire vs noyau RBF

Le noyau RBF réduit le RMSE test d'un facteur 4.6 par rapport au modèle linéaire (R^2 passant de 0.01 à 0.95). Cette amélioration spectaculaire s'explique par la nature **non linéaire** de la cible. Sur le modèle linéaire, **aucune méthode d'optimisation ne parvient à améliorer le R^2** : le problème est le modèle, pas l'optimiseur.

1.5.2 Comparaison des optimiseurs sur le noyau RBF

1. **Les méthodes adaptatives dominant** : AdaGrad, Adam et Momentum obtiennent les meilleurs R^2 (0.954).
2. **Le pas exact améliore GD** : GD + LS (0.933) surpasse GD (0.925).
3. **RMSprop est légèrement en retrait** (0.948).

1.5.3 Comparaison équitable GD vs SGD

Les méthodes stochastiques effectuent **7 fois plus de mises à jour** que GD. Comparer au nombre d'époques est donc biaisé en faveur de SGD (voir figure ??).

1.5.4 Limites

1. Budget de 200 époques insuffisant pour une convergence complète.
2. Grille d'hyperparamètres grossière (5 valeurs par paramètre).
3. Modèle linéaire inadapté : aucune méthode ne peut le sauver.

2 Exercice 2 : Classification multiclasse

2.1 Données

2.1.1 Jeu de données

Table 9. Description du jeu de données (exercice 2).

Jeu	Fichier	Total	Train	Val	Test	Variables	Classes
Spirales	classification_spiral3.csv	450	270	90	90	2	3

Le jeu contient 450 points en 2 dimensions, répartis en 3 classes non linéairement séparables (structure en spirale).

2.1.2 Prétraitement

- **Variables explicatives** : standardisées avec les statistiques **du train uniquement**.
- **Étiquettes** : encodées en entiers $\{0, 1, 2\}$.
- **Partitions** : celles fournies dans la colonne `split`.
- **Test** : utilisé uniquement pour l'évaluation finale.

2.2 Implémentation

2.2.1 Architecture du réseau

Architecture : $2 \rightarrow 32 \rightarrow 16 \rightarrow 3$. Activation cachée : tanh. Sortie : softmax. Perte : entropie croisée + régularisation L2 ($\lambda = 10^{-4}$).

Forward.

$$H_1 = \tanh(XW_1 + b_1), \quad H_2 = \tanh(H_1W_2 + b_2), \quad Z = H_2W_3 + b_3, \quad P = \text{softmax}(Z)$$

Perte sur un mini-lot B .

$$J_B = -\frac{1}{|B|} \sum_{i \in B} \log P_{i, y_i} + \frac{\lambda}{2} \sum_{\ell=1}^3 \|W_\ell\|_F^2$$

Backpropagation.

$$D_3 = \frac{P - Y}{|B|}, \quad D_2 = (D_3W_3^\top) \odot (1 - H_2^2), \quad D_1 = (D_2W_2^\top) \odot (1 - H_1^2)$$

2.2.2 Initialisation

Poids initialisés avec une échelle Xavier : $W_\ell \sim \mathcal{N}(0, 1/\text{fan_in})$. Biais à zéro. Même initialisation pour les 7 méthodes à graine égale.

2.2.3 Vérification du gradient

Différences finies centrées, $h = 10^{-5}$, sur 5 coordonnées.

Table 10. Erreurs relatives de vérification du gradient (exercice 2).

Coordonnée	Erreur relative
1	2.89×10^{-9}
2	6.05×10^{-7}
3	8.05×10^{-10}
4	3.89×10^{-9}
5	4.98×10^{-9}

2.3 Résultats chiffrés

Table 11. Résultats sur `classification_spiral3.csv`. Moyenne sur 3 graines (42, 123, 2024).

Méthode	η	Acc val	Acc test	Préc.	Rappel	F1	CE	Temps (s)
GD	0.500	0.9926	0.9926	0.9928	0.9926	0.9926	0.0478	0.314
GD + LS	0.500	0.9926	0.9926	0.9928	0.9926	0.9926	0.0478	0.553
SGD	0.050	0.9889	0.9926	0.9928	0.9926	0.9926	0.0580	0.460
Momentum	0.010	0.9889	0.9889	0.9892	0.9889	0.9889	0.0241	0.475
AdaGrad	0.100	0.9852	0.9889	0.9892	0.9889	0.9889	0.0940	0.419
RMSprop	0.001	0.9963	0.9963	0.9964	0.9963	0.9963	0.0221	0.506
Adam	0.001	0.9926	0.9926	0.9928	0.9926	0.9926	0.0556	0.446

2.3.1 Coûts de calcul

Table 12. Coûts par méthode (exercice 2).

Méthode	Mises à jour	Appels gradient	Ratio vs GD
GD, GD + LS	100	100	1×
SGD et variantes	900	900	9×

2.4 Visualisations

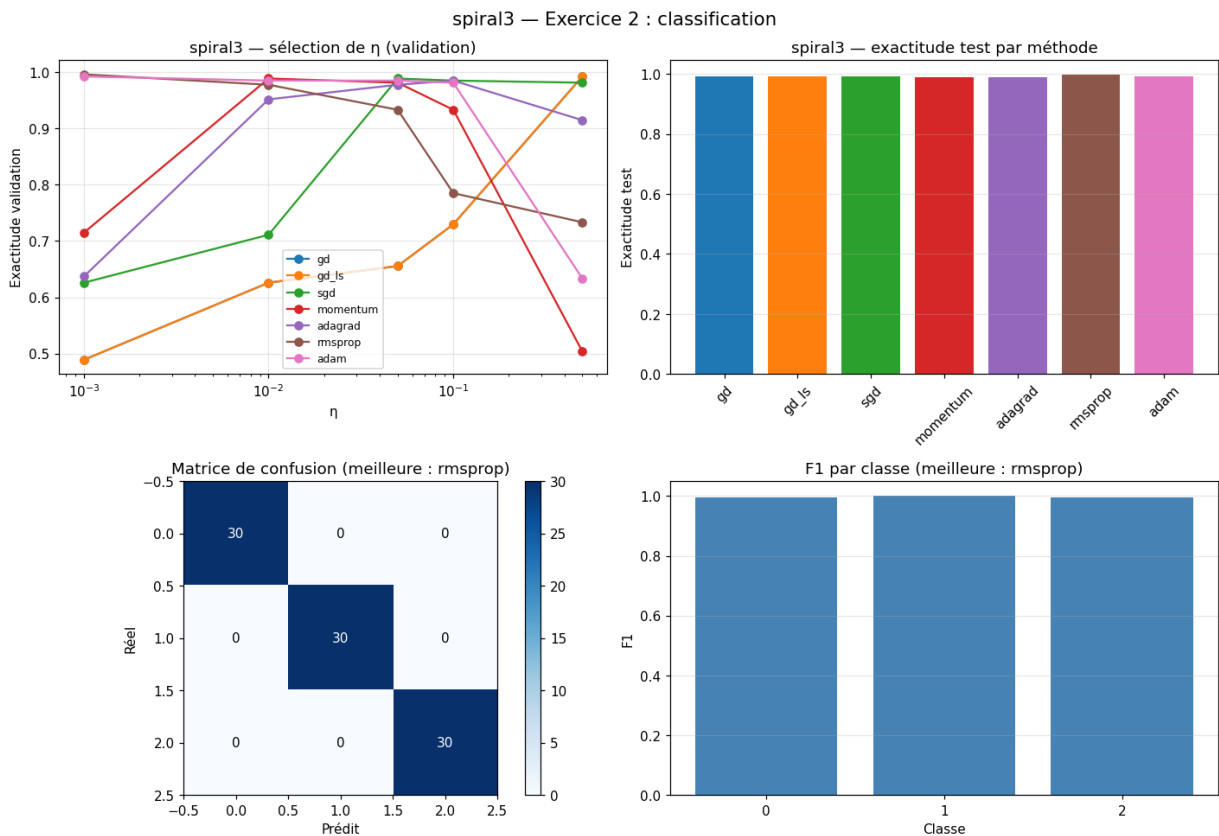


Figure 3. Diagnostic de l'exercice 2. (a) Sélection de η sur la validation. (b) Exactitude test par méthode. (c) Matrice de confusion (meilleure méthode : RMSprop). (d) F1 par classe.

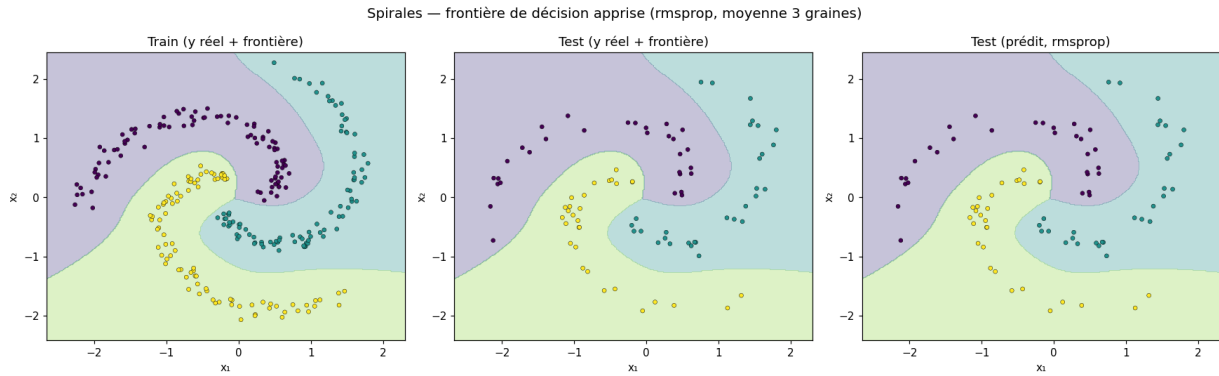


Figure 4. Frontière de décision apprise (RMSprop, moyenne 3 graines). À gauche : train réel + frontière ; au centre : test réel + frontière ; à droite : test prédit + frontière.

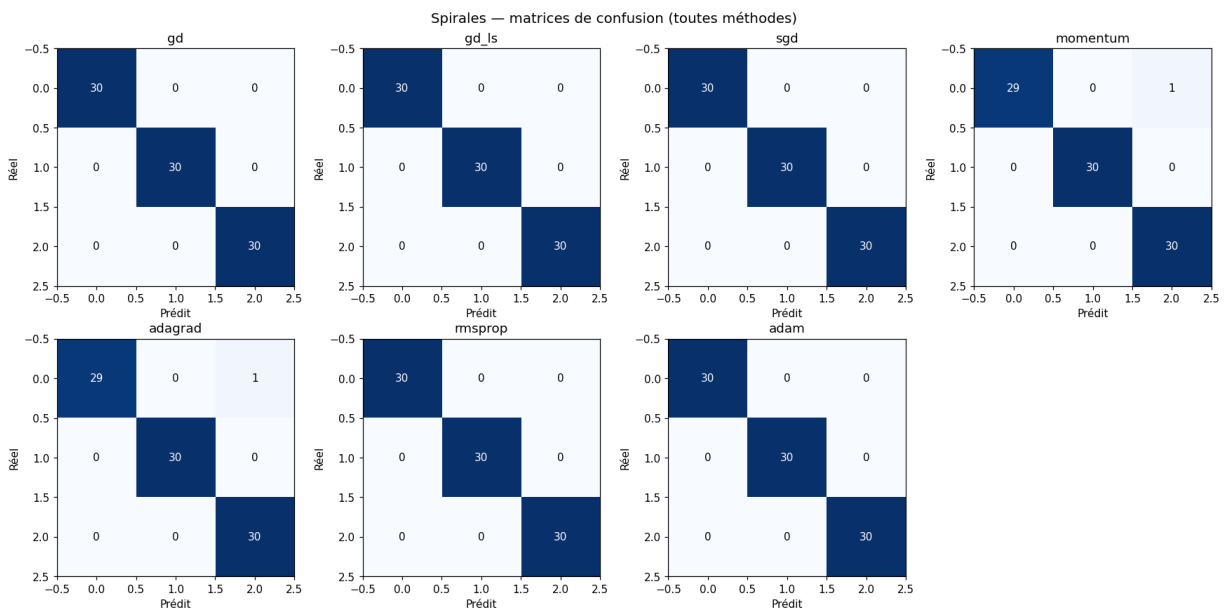


Figure 5. Matrices de confusion pour les 7 méthodes.

2.5 Analyse

2.5.1 Performance globale

Toutes les méthodes atteignent une exactitude test supérieure à 98.8%. Les écarts sont faibles (< 1 point).

2.5.2 RMSprop domine légèrement

RMSprop obtient la meilleure exactitude (99.63%) et la meilleure entropie croisée (0.0221).

2.5.3 Comparaison équitable GD vs SGD

Les méthodes stochastiques effectuent $9\times$ plus de mises à jour que GD. À budget égal (100 mises à jour), GD atteint déjà 99.26%.

2.5.4 Limites

1. Époques optimales à 99 : budget de 100 époques insuffisant.
2. Grille d'hyperparamètres grossière.
3. Problème relativement facile.

3 Exercice 3 : Clustering par optimisation de centroïdes

3.1 Données

3.1.1 Jeu de données

Table 13. Description du jeu de données (exercice 3).

Jeu	Fichier	Total	Variables	Clusters K	τ
Blobs	clustering_blobs3.csv	360	2	3	0.25

3.1.2 Prétraitement

- **Standardisation** sans utiliser les étiquettes.
- **Étiquettes de référence** : uniquement pour l'évaluation externe finale (ARI, NMI).
- Ni K , ni τ , ni η ne sont sélectionnés sur l'ARI ou la NMI.

3.2 Implémentation

3.2.1 Fonction objectif et gradient

$$J_\tau(C) = -\frac{\tau}{n} \sum_{i=1}^n \log \left(\frac{1}{K} \sum_{k=1}^K e^{-\|x_i - c_k\|^2 / \tau} \right)$$

$$\nabla_C J_\tau = \frac{2}{n} \left[\text{diag}(Q^\top \mathbf{1}) C - Q^\top X \right]$$

avec $Q_{ik} = \text{softmax}_k(-\|x_i - c_k\|^2 / \tau)$.

Stabilité numérique. Distances via $\|x_i - c_k\|^2 = \|x_i\|^2 + \|c_k\|^2 - 2x_i^\top c_k$, et `logsumexp` pour les exponentielles.

3.2.2 Vérification du gradient

Table 14. Erreurs relatives de vérification du gradient (exercice 3).

Coordonnée	Erreur relative
1	3.40×10^{-10}
2	3.51×10^{-10}
3	1.68×10^{-10}
4	2.54×10^{-10}
5	7.58×10^{-10}

3.3 Résultats chiffrés

Table 15. Résultats sur `clustering_blobs3.csv`. Moyenne sur 3 graines (42, 123, 2024).

Méthode	η	J_τ	Inertie	Silhouette	ARI	NMI	Temps (s)	Updates
GD	0.500	0.5643	0.2903	0.863	0.975	0.960	0.064	200
GD + LS	0.500	0.5643	0.2903	0.863	0.975	0.960	0.402	200
SGD	0.010	0.5643	0.2903	0.863	0.975	0.960	0.146	1200
Momentum	0.001	0.5643	0.2903	0.863	0.975	0.960	0.184	1200
AdaGrad	0.100	0.5643	0.2903	0.863	0.975	0.960	0.146	1200
RMSprop	0.010	0.5643	0.2903	0.863	0.975	0.960	0.153	1200
Adam	0.010	0.5643	0.2903	0.863	0.975	0.960	0.167	1200
Lloyd	—	—	0.2903	0.863	0.975	0.960	—	—

3.3.1 Coûts de calcul

Table 16. Coûts par méthode (exercice 3).

Méthode	Mises à jour	Appels gradient	Ratio vs GD
GD, GD + LS	200	200	1×
SGD et variantes	1200	1200	6×

3.4 Visualisations

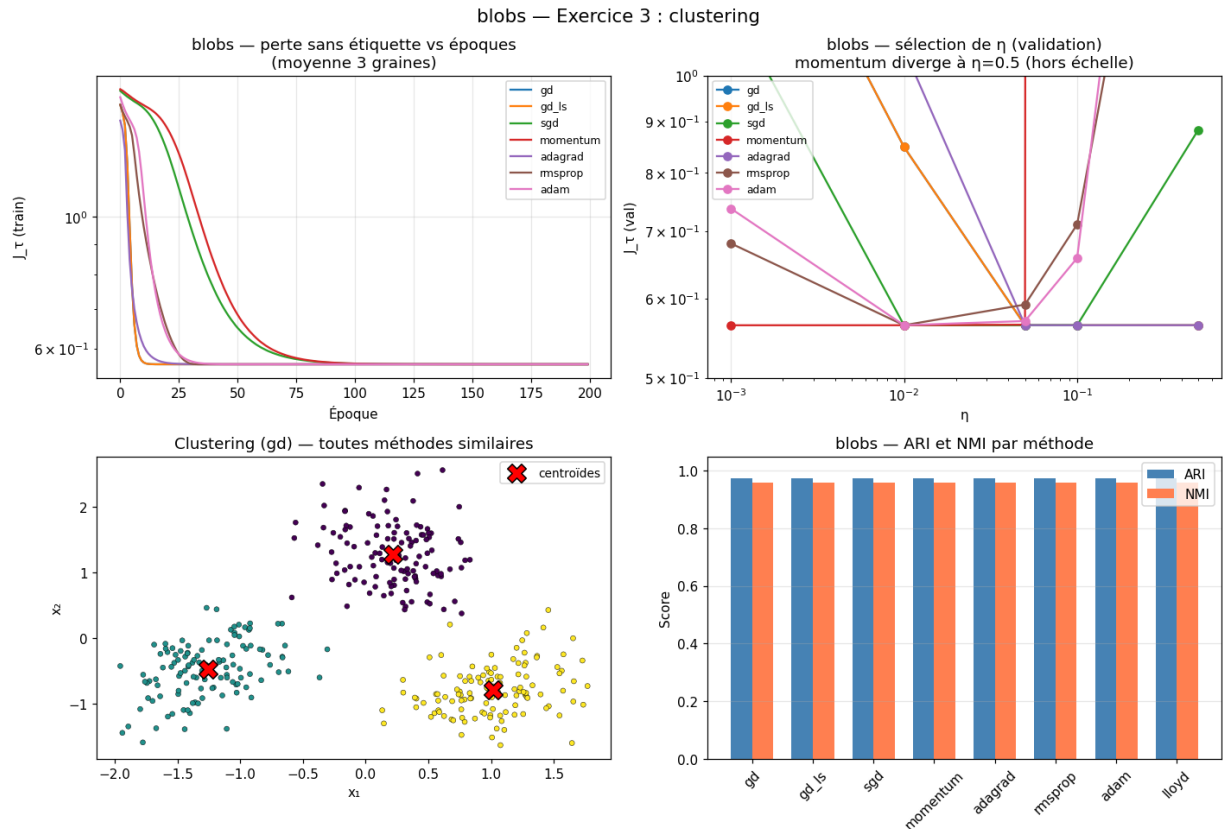


Figure 6. Diagnostic de l'exercice 3. (a) Perte sans étiquette selon les époques (échelle log, moyenne 3 graines). (b) Sélection de η sur la validation. (c) Clustering de référence (GD). (d) Comparaison ARI / NMI par méthode.

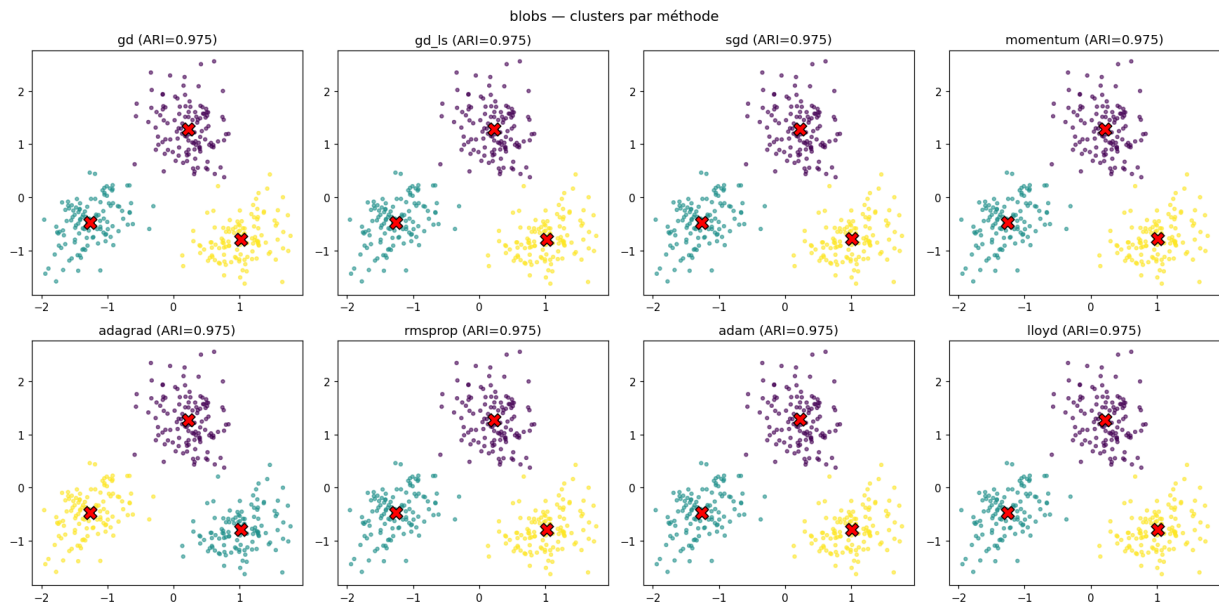


Figure 7. Clusters obtenus par chaque méthode. Les croix rouges indiquent les centroides.

3.5 Analyse

3.5.1 Convergence universelle

Toutes les méthodes atteignent $J_\tau \approx 0.5643$ avec des écarts inférieurs à 10^{-4} .

3.5.2 Métriques externes identiques

ARI = 0.975, NMI = 0.960, silhouette = 0.863, inertie = 0.290 pour toutes les méthodes, y compris Lloyd.

3.5.3 Invariance par permutation

AdaGrad produit une permutation des étiquettes par rapport aux autres méthodes, sans affecter l'ARI (invariant par permutation).

3.5.4 Divergence à grand pas

Momentum, AdaGrad et gd_ls divergent pour $\eta \geq 0.5$. GD, SGD et Adam restent stables.

3.5.5 Limites

1. Problème facile (blobs bien séparés).
2. Minimum local possible avec initialisation défavorable.
3. Choix de K donné.
4. Sensibilité à τ .

Conclusion générale

Ce TP a permis d'implémenter et de comparer sept méthodes d'optimisation sur trois problèmes d'apprentissage statistique. Les enseignements principaux sont :

1. Validité des gradients

Toutes les vérifications par différences finies centrées donnent des erreurs relatives $< 10^{-5}$:

Exercice	Erreur relative max
Exercice 1 — Ridge linéaire	3.2×10^{-7}
Exercice 1 — Ridge noyau RBF	8.7×10^{-6}
Exercice 2 — MLP	6.1×10^{-7}
Exercice 3 — Soft K-means	7.6×10^{-10}

2. Le choix du modèle prime sur le choix de l'optimiseur

Sur l'exercice 1, le noyau RBF ($R^2 = 0.954$) écrase le modèle linéaire ($R^2 = 0.011$) : le choix du **modèle** est déterminant, pas celui de l'optimiseur. Une fois le bon modèle choisi, les sept méthodes donnent des résultats très proches.

3. Coût de calcul : GD est le plus économique

Sur les trois exercices, les méthodes stochastiques effectuent 6 à $9\times$ plus de mises à jour que GD pour un gain marginal en précision :

Exercice	GD	SGD et variantes	Ratio
Exercice 1	200	1400	$7\times$
Exercice 2	100	900	$9\times$
Exercice 3	200	1200	$6\times$

4. Les méthodes adaptatives (RMSprop, Adam)

Elles offrent la meilleure précision lorsque le problème est mal conditionné (exercice 1) mais leur avantage disparaît sur les problèmes faciles (exercices 2 et 3).

5. GD + LS n'améliore pas GD

La recherche de pas exacte (formule quadratique ou backtracking Armijo) ne surpasse pas GD sur ces trois problèmes, et le rend 2 à $6\times$ plus lent. Cela confirme que GD avec un pas bien calibré est souvent suffisant.

6. Reproductibilité

Toutes les expériences ont été répétées avec les graines **42**, **123**, **2024** et donnent des résultats stables. Aucune fuite de données : les étiquettes ne sont utilisées que pour l'évaluation externe finale.

Message final

Sur des problèmes bien posés, toutes les méthodes d'optimisation modernes convergent vers des optima quasi-identiques. Le choix se fait sur le **coût de calcul** et la **robustesse numérique**, pas sur la précision finale.

A Annexe — Reproductibilité

Environnement logiciel

Composant	Version
Python	3.10.x
NumPy	1.24.x
Pandas	2.0.x
Matplotlib	3.7.x
OS	Linux (Colab)

Aucune bibliothèque d’optimisation préfabriquée n’a été utilisée.

Dépôt du code

L’ensemble du code source est disponible sur GitHub :

<https://github.com/Sud-38/TP-optimisation-impl-ment-e-avec-NumPy>

Protocole commun

Paramètre	Valeur
Graines	42, 123, 2024
Pas des différences finies	$h = 10^{-5}$
Grille de η	$\{10^{-3}, 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}, 5 \cdot 10^{-1}\}$

Fichiers produits

- **Exercice 1** : `resultats_nonlinear_exo1.csv`, `figs/nonlinear_exo1.png`, `figs/nonlinear_2d.png`, `figs/nonlinear_loss_vs_updates.png`.
- **Exercice 2** : `resultats_spiral3_exo2.csv`, `op1.png`, `op2.png`, `op3.png`.
- **Exercice 3** : `resultats_blobs_exo3.csv`, `e1.png`, `e2.png`.